

Manual for Artificial Intelligence and Applications

Lab BCA VI Sem – Final AI Lab Programs

– Arshad Pasha

All Code , For Studying 🤪 :

<https://github.com/pashaarshad/AI-LAB-6th-Sem-2026>

1. Write a program to implement breadth first search using python.

```
2. g={
3.     'A':['B','C'],
4.     'B':['D','E'],
5.     'C':['F'],
6.     'D':[],
7.     'E':['F'],
8.     'F':[]
9. }
10.
11. v = []
12. q = []
13.
14. def bfs(v,g,n):
15.     v.append(n)
16.     q.append(n)
17.
18.     while q:
19.         s=q.pop(0)
20.         print(s,end=" ")
21.
22.         for nib in g[s]:
23.             if nib not in v:
24.                 v.append(nib)
25.                 q.append(nib)
26. bfs(v,g,'A')
27.
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p1.py
ABCDEF
```

2. Write a Program to Implement Depth First Search using Python.

```
28. g={
29.     'A':['B','C'],
30.     'B':['D','E'],
31.     'C':['F'],
32.     'D':[],
33.     'E':['F'],
34.     'F':[]
35. }
36. v = set()
37.
38. def dfs(v,g,n):
39.     if n not in v:
40.         print(n,end="")
41.         v.add(n)
42.
43.         for n in g[n]:
44.             dfs(v,g,n)
45.
46. dfs(v,g,'A')
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p2.py
ABDEFC
```

3. . Write a Program to Implement 8-Puzzle problem using Python.

```
47. # -----
48. # Initial and Goal States
49. # -----
50.
51. start = [1, 2, 3,
52.         4, 0, 6,
53.         7, 5, 8]
54.
55. goal = [1, 2, 3,
56.        4, 5, 6,
57.        7, 8, 0]
58.
59. # -----
60. # Possible moves (index positions)
61. # -----
62.
63. moves = {
64.     0: [1, 3],
65.     1: [0, 2, 4],
66.     2: [1, 5],
67.     3: [0, 4, 6],
68.     4: [1, 3, 5, 7],
69.     5: [2, 4, 8],
70.     6: [3, 7],
71.     7: [4, 6, 8],
72.     8: [5, 7]
73. }
74.
75. # -----
76. # BFS Solve Function
77. # -----
78.
79. def solve(start):
80.     visited = []
81.     queue = []
82.
83.     queue.append((start, [])) # (current_state, path)
84.     visited.append(start)
85.
86.     while queue:
87.         current, path = queue.pop(0)
88.
89.         # Check if goal reached
90.         if current == goal:
91.             return path + [current]
92.
93.         zero_index = current.index(0)
94.
95.         # Try all possible moves
96.         for move in moves[zero_index]:
97.             new_state = current.copy()
98.
99.             # Swap 0 with neighbor
```

```

100.     new_state[zero_index], new_state[move] = new_state[move], new_state[zero_index]
101.
102.     if new_state not in visited:
103.         visited.append(new_state)
104.         queue.append((new_state, path + [current]))
105.
106.     return None
107.
108. # -----
109. # Print Puzzle Function
110. # -----
111.
112. def print_puzzle(state):
113.     # state is a list like [1,2,3,4,5,6,7,8,0]
114.
115.     for i in range(0, 9, 3):
116.         # range(0, 9, 3) gives: 0, 3, 6
117.
118.         # state[i:i+3] means:
119.         # start from index i
120.         # go up to (but not including) i+3
121.
122.         # Example:
123.         # i = 0 → state[0:3] → [1, 2, 3]
124.         # i = 3 → state[3:6] → [4, 5, 6]
125.         # i = 6 → state[6:9] → [7, 8, 0]
126.
127.         print(state[i:i+3]) # prints one row (3 elements)
128.
129.     print() # empty line for spacing
130.
131. # -----
132. # Main Execution
133. # -----
134.
135. solution = solve(start)
136.
137. if solution:
138.     print("Solution Found!\n")
139.
140.     for step in solution:
141.         print_puzzle(step)
142.
143.     print("Total Moves:", len(solution) - 1)
144.
145. else:
146.     print("No Solution")

```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p3.py  
Solution Found!
```

```
[1, 2, 3]  
[4, 0, 6]  
[7, 5, 8]
```

```
[1, 2, 3]  
[4, 5, 6]  
[7, 0, 8]
```

```
[1, 2, 3]  
[4, 5, 6]  
[7, 8, 0]
```

```
Total Moves: 2
```

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI>
```

4. Write a Program to Implement Alpha-Beta Pruning using Python.

```
147.
148. # Alpha-Beta Pruning in Python
149.
150. MAX, MIN = 1000, -1000
151.
152. def minimax(depth, nodeIndex, maximizingPlayer,
153.             values, alpha, beta):
154.
155.     # Leaf node reached
156.     if depth == 3:
157.         return values[nodeIndex]
158.
159.     # Maximizer
160.     if maximizingPlayer:
161.
162.         best = MIN
163.
164.         for i in range(2):
165.
166.             val = minimax(depth + 1,
167.                           nodeIndex * 2 + i,
168.                           False, values,
169.                           alpha, beta)
170.
171.             best = max(best, val)
172.             alpha = max(alpha, best)
173.
174.             # Pruning
175.             if beta <= alpha:
176.                 break
177.
178.         return best
179.
180.     # Minimizer
181.     else:
182.
183.         best = MAX
184.
185.         for i in range(2):
186.
187.             val = minimax(depth + 1,
188.                           nodeIndex * 2 + i,
189.                           True, values,
190.                           alpha, beta)
191.
```

```

192.         best = min(best, val)
193.         beta = min(beta, best)
194.
195.         # Pruning
196.         if beta <= alpha:
197.             break
198.
199.     return best
200.
201. # Terminal values
202. values = [3, 5, 6, 9, 1, 2, 0, -1]
203.
204. # Function Call
205. result = minimax(0, 0, True,
206.                  values, MIN, MAX)
207.
208. print("Optimal Value is :", result)
209.

```

Output :

```

PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p5.py
Optimal Value is : 5

```

5. Write a program to implement forward chaining algorithm.

```
210. # Forward Chaining using ABCD
211.
212. database = ["A", "B", "C", "D"]
213.
214. def display():
215.
216.     print("\n1. A")
217.     print("2. B")
218.     print("3. C")
219.     print("4. D")
220.
221.     print("\nSelect One :", end=' ')
222.
223. def main():
224.
225.     print("*----- Forward Chaining -----*")
226.
227.     display()
228.
229.
230.     x = int(input())
231.
232.     print()
233.
234.     # Rules
235.     if x == 1:
236.         print("A -> B")
237.
238.     elif x == 2:
239.         print("B -> C")
240.
241.     elif x == 3:
242.         print("C -> D")
243.
244.     elif x == 4:
245.         print("Goal Reached : D")
246.
247.     else:
248.         print("Invalid Option")
249.
250.     # Display selected value
251.     if x >= 1 and x <= 4:
252.
253.         print("\nSelected :", database[x - 1])
```



```
254.  
255. if __name__ == "__main__":  
256.     main()  
257.
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p6.py  
*----- Forward Chaining -----*  
  
1. A  
2. B  
3. C  
4. D  
  
Select One : 1  
  
A -> B  
  
Selected : A
```

6. Write a program to implement backward chaining algorithm.

```
258. # Backward Chaining using ABCD
259.
260. database = ["A", "B", "C", "D"]
261.
262. def main():
263.
264.     print("*----- Backward Chaining -----*")
265.
266.     goal = input("\nEnter Goal : ")
267.
268.     print()
269.
270.     # Backward Rules
271.     if goal == "D":
272.
273.         print("D <- C")
274.         print("C <- B")
275.         print("B <- A")
276.
277.         print("\nGoal Achieved")
278.
279.     elif goal == "C":
280.
281.         print("C <- B")
282.         print("B <- A")
283.
284.         print("\nGoal Achieved")
285.
286.     elif goal == "B":
287.
288.         print("B <- A")
289.
290.         print("\nGoal Achieved")
291.
292.     elif goal == "A":
293.
294.         print("A is the Initial Fact")
295.
296.     else:
297.
298.         print("Invalid Goal")
299. if __name__ == "__main__":
300.     main()
301.
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p7.py
*----- Backward Chaining -----*

Enter Goal : D

D <- C
C <- B
B <- A

Goal Achieved
```

7. Write a program to implement KNN algorithm to classify Iris dataset. Print both correct and wrong predictions.

DataSet Link : <https://github.com/pashaarshad/BCA-6sem/blob/main/AI/Iris.csv>

```
302.
303.
304.
305. # KNN Algorithm using Iris CSV Dataset
306.
307. import pandas as pd
308.
309. from sklearn.model_selection import train_test_split
310. from sklearn.neighbors import KNeighborsClassifier
311. from sklearn.metrics import accuracy_score
312.
313. # Load Dataset
314. data = pd.read_csv("Iris.csv")
315.
316. # Input and Output
317. X = data.iloc[:, 1:-1].values
318. y = data.iloc[:, -1].values
319.
320. # Split Dataset
321. X_train, X_test, y_train, y_test = train_test_split(
322.     X, y, test_size=0.2, random_state=1)
323.
324. # Create Model
325. model = KNeighborsClassifier(n_neighbors=3)
326.
327. # Train Model
328. model.fit(X_train, y_train)
329.
330. # Prediction
331. pred = model.predict(X_test)
332.
333. # Accuracy
334. print("Accuracy :", accuracy_score(y_test, pred))
335.
336. print("\nPredictions:\n")
337.
338. # Print Correct and Wrong Predictions
339. for i in range(len(y_test)):
340.
341.     print("Actual :", y_test[i],
342.         " Predicted :", pred[i], end=' ')
```

```
343.  
344.     if y_test[i] == pred[i]:  
345.         print("-> Correct")  
346.  
347.     else:  
348.         print("-> Wrong")
```

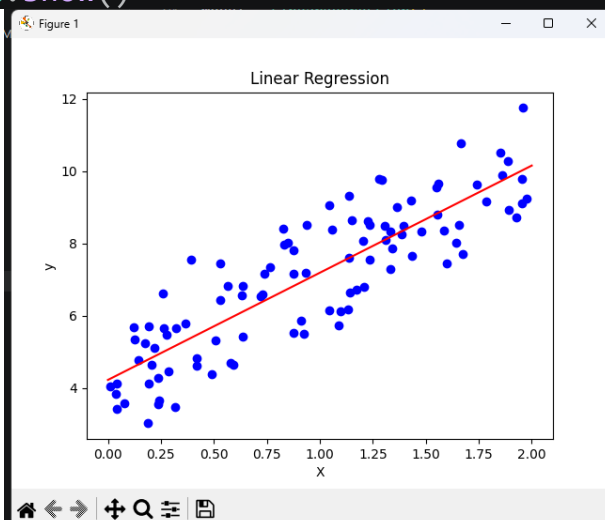
Output :

```
● PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users  
Accuracy : 0.9666666666666667  
  
Predictions:  
  
Actual : setosa Predicted : setosa -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : virginica -> Wrong  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct
```

8. Train a random data sample using linear regression model and plot the graph.

DataSet Link : <https://github.com/pashaarshad/BCA-6sem/blob/main/AI/Iris.csv>

```
349.  
350. import numpy as np  
351. import matplotlib.pyplot as plt  
352.  
353. from sklearn.linear_model import LinearRegression  
354.  
355. # Generate some sample data  
356. np.random.seed(0)  
357.  
358. X = 2 * np.random.rand(100, 1)  
359. y = 4 + 3 * X + np.random.randn(100, 1)  
360.  
361. # Train the linear regression model  
362. model = LinearRegression()  
363. model.fit(X, y)  
  
364. # Make predictions  
365. X_new = np.array([[0], [2]])  
366. y_pred = model.predict(X_new)  
367.  
368. # Plot the data points and the linear regression line  
369.  
370. plt.scatter(X, y, color='blue')  
371. plt.plot(X_new, y_pred, color='red')  
372. plt.xlabel('X')  
373. plt.ylabel('y')  
374. plt.title('Linear Regression')  
375.  
376. plt.show()
```



Output :

9. Implement the naïve Bayesian classifier for a sample training data set stored as a .csv file. Compute the accuracy of the classifier, considering few test data sets.

DataSet Link : <https://github.com/pashaarshad/BCA-6sem/blob/main/AI/Iris.csv>

```
377. # Naive Bayes using Iris CSV Dataset
378.
379. import pandas as pd
380.
381. from sklearn.model_selection import train_test_split
382. from sklearn.naive_bayes import GaussianNB
383. from sklearn.metrics import accuracy_score, confusion_matrix
384.
385. # Load Dataset
386. data = pd.read_csv("Iris.csv")
387.
388. # Input and Output
389. X = data.iloc[:, 1:-1].values
390. y = data.iloc[:, -1].values
391.
392. # Split Dataset
393. X_train, X_test, y_train, y_test = train_test_split(
394.     X, y, test_size=0.2, random_state=1)
395.
396. # Create Model
397. model = GaussianNB()
398.
399. # Train Model
400. model.fit(X_train, y_train)
401.
402. # Prediction
403. pred = model.predict(X_test)
404.
405. # Accuracy
406. print("Accuracy :", accuracy_score(y_test, pred))
407.
408. # Confusion Matrix
409. print("\nConfusion Matrix:\n")
410.
411. print(confusion_matrix(y_test, pred))
412.
```

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "python Iris_classifier.py"
Accuracy : 0.9666666666666667

Confusion Matrix:

[[11  0  0]
 [ 0 12  1]
 [ 0  0  6]]
```

Output :

10. Demonstrate the working of SVM classifier for a suitable data set(e.g., iris dataset)

```
413. # SVM Classifier using Iris CSV Dataset
414.
415. import pandas as pd
416.
417. from sklearn.model_selection import train_test_split
418. from sklearn.preprocessing import StandardScaler
419.
420. from sklearn.svm import SVC
421. from sklearn.metrics import accuracy_score, confusion_matrix
422.
423. # Load Dataset
424. data = pd.read_csv("Iris.csv")
425.
426. # Input and Output
427. X = data.iloc[:, 1:-1].values
428. y = data.iloc[:, -1].values
429.
430. # Split Dataset
431. X_train, X_test, y_train, y_test = train_test_split(
432.     X, y, test_size=0.2, random_state=42)
433.
434. # Feature Scaling
435. # Below 3 lines are optional
436.
437. scaler = StandardScaler()
438.
439. X_train = scaler.fit_transform(X_train)
440. X_test = scaler.transform(X_test)
441.
442. # Create SVM Model
443. model = SVC(kernel='linear')
444.
445. # Train Model
446. model.fit(X_train, y_train)
447.
448. # Prediction
449. pred = model.predict(X_test)
450.
451. # Accuracy
452. print("Accuracy :", accuracy_score(y_test, pred))
453. # Confusion Matrix
454. print("\nConfusion Matrix:\n")
455.
456. print(confusion_matrix(y_test, pred))
457.
```


Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python311\python.exe" C:\Users\Admin\Desktop\BCA-6sem\AI\main.py
Accuracy : 0.9666666666666667

Confusion Matrix:

[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
PS C:\Users\Admin\Desktop\BCA-6sem\AI>
```